

Báo cáo Nghiên cứu Chuyên sâu Toàn diện: Agent Skills KIT

Mục lục nhanh

- 1. Tổng quan Dự án
 - 2. Kiến trúc Hệ thống
 - 3. Meta-Skill: `using-agent-skills`
 - 4. Hệ thống 21 Kỹ năng (20+1)
 - 5. Giải phẫu một Kỹ năng
 - 6. Phân tích 7 Lệnh
 - 7. Hệ thống Personas
 - 8. Orchestration Patterns
 - 9. Hooks System
 - 10. Tương thích Đa nền tảng
 - 10.1. Cài đặt với Cursor
 - 10.2. Cài đặt với Gemini CLI
 - 10.3. Các nền tảng khác
 - 11. Tổng kết Đánh giá
-

1. Tổng quan Dự án (Project Overview)

1.1. Khái niệm và Mục đích cốt lõi

Agent Skills là một bộ công cụ cung cấp các "kỹ năng kỹ thuật chuẩn sản xuất" (production-grade engineering skills) dành riêng cho các AI Coding Agents (Claude Code, Cursor, Copilot, OpenCode, Windsurf, Gemini CLI...). Hệ thống được cấu trúc quanh **7 lệnh slash commands** chính và **21 kỹ năng (20 core + 1 meta)** chuyên sâu.

Vấn đề lớn nhất của các AI Agent hiện nay là **xu hướng chọn "con đường ngắn nhất"** – bỏ qua viết tài liệu, kiểm thử, đánh giá bảo mật. Agent Skills giải quyết triệt để bằng cách ép AI tuân thủ workflows, quality gates và best practices từ văn hóa kỹ sư Google (TDD, Chesterton's Fence, trunk-based development). (Nguồn: `README.md`, dòng 1-5, 278-282)

1.2. Triết lý Thiết kế (Design Principles)

4 nguyên tắc cốt lõi:

- **Process, not prose:** Kỹ năng là workflows AI bắt buộc phải theo, không phải văn xuôi để "tham khảo". Mỗi kỹ năng có bước, điểm kiểm tra và tiêu chí thoát rõ ràng.
- **Anti-rationalization:** Mọi kỹ năng chứa bằng ngụy biện phổ biến AI hay dùng (như "tôi sẽ thêm test sau") kèm lý lẽ phản bác.
- **Verification is non-negotiable:** Luôn yêu cầu bằng chứng thực tế (test passing, build output, runtime data). "Có vẻ đúng" là không đủ.
- **Progressive disclosure:** Chỉ load thông tin tổng quan lúc đầu, references load khi thực sự cần, tiết kiệm token. (Nguồn: `README.md`, dòng 231-237)

1.3. Sáu Hành vi Bắt buộc Xuyên suốt (Core Operating Behaviors)

Đây là 6 quy tắc **phi thương lượng** áp dụng cho mọi skill:

1. **Surface Assumptions** — Nêu rõ giả định trước khi code. Không âm thầm tự quyết.
2. **Manage Confusion Actively** — DỪNG NGAY khi gặp mâu thuẫn. Nêu cụ thể điểm rối. Chờ giải quyết.
3. **Push Back When Warranted** — AI phải phản đối nếu cách tiếp cận sai. "Sycophancy is a failure mode."
4. **Enforce Simplicity** — Nếu build 1000 dòng mà 100 đã đủ, đó là thất bại. Ưu tiên giải pháp đơn giản.
5. **Maintain Scope Discipline** — Chỉ chạm vào phần được yêu cầu. Không "dọn dẹp" code không liên quan.
6. **Verify, Don't Assume** — Mọi task chưa hoàn thành cho đến khi có bằng chứng xác minh. (Nguồn: `skills/using-agent-skills/SKILL.md`, dòng 39-107)

2. Kiến trúc Hệ thống (Architecture)

2.1. Cấu trúc Phân tầng (3 Orchestration Layers)

Tầng	Đường dẫn	Vai trò	Ví dụ
Skills (The How)	<code>skills/<name>/SKILL.md</code>	Quy trình bắt buộc khi intent khớp	<code>code-review-and-quality</code>
Personas (The Who)	<code>agents/<role>.md</code>	Vai trò + góc nhìn + định dạng output	<code>code-reviewer</code>

Tầng	Đường dẫn	Vai trò	Ví dụ
Commands (The When)	<code>.claude/commands/*.md</code>	Điều phối, gọi skills/personas	<code>/review</code> , <code>/ship</code>

Quy tắc vàng: User (hoặc slash command) là orchestrator. **Personas KHÔNG ĐƯỢC gọi personas khác.** Skills là hops bắt buộc bên trong persona. (Nguồn: `AGENTS.md`, dòng 68-80; `agents/README.md`, dòng 21)

2.2. Cấu trúc Thư mục

- `skills/` : **21 thư mục** — 20 kỹ năng kỹ thuật + 1 meta-skill (`using-agent-skills`).
- `agents/` : 3 persona chuyên gia + README hướng dẫn.
- `references/` : **5 file** tài liệu bổ trợ (Testing, Security, Performance, Accessibility, **Orchestration Patterns**).
- `hooks/` : Hệ thống hook vòng đời session (Session Start, SDD-Cache, Simplify-Ignore).
- `.claude/commands/` và `.gemini/commands/` : 7 slash commands cho mỗi nền tảng.
- `docs/` : Tập tài liệu hỗn hợp, gồm setup guides cho một số nền tảng chính và tài liệu dùng chung như `getting-started.md`, `skill-anatomy.md`.

Cây cấu trúc rút gọn của bộ KIT:

```

agent-skills/
├── README.md           # Giới thiệu tổng quan, quick start, danh sách skills/c
├── AGENTS.md           # Luật orchestration và cách map intent → skill
├── CLAUDE.md           # Hướng dẫn riêng khi dùng với Claude/Claude Code
├── CONTRIBUTING.md     # Quy tắc đóng góp, viết thêm skill mới
├── LICENSE             # Giấy phép sử dụng
├── .claude/
│   └── commands/       # 7 slash commands cho Claude Code: /spec, /plan, /buil
├── .gemini/
│   └── commands/       # Bộ command tương đương cho Gemini CLI dưới dạng .toml
├── .opencode/
│   └── skills/         # Điểm tích hợp để OpenCode load skills
├── .github/
│   └── workflows/      # CI kiểm tra plugin/cài đặt và workflow liên quan
├── .claude-plugin/     # Metadata/plugin packaging cho Claude Code marketplace
├── skills/
│   ├── using-agent-skills/ # Meta-skill: chọn đúng skill theo loại task
│   ├── spec-driven-development/ # Skill viết spec trước khi code
│   └── planning-and-task-breakdown/ # Skill chia việc thành vertical slices

```

```

|   └─ incremental-implementation/ # Skill build từng lát nhỏ, có verify
|   └─ ...                         # Các skill còn lại cho test, review, ship, security ...
└─ agents/
|   └─ code-reviewer.md           # Persona review code theo 5 trục
|   └─ security-auditor.md        # Persona audit security
|   └─ test-engineer.md           # Persona đánh giá test strategy/coverage
|   └─ README.md                  # Giải thích vai trò và quy tắc composition của persona
└─ references/
|   └─ testing-patterns.md        # Checklist/pattern cho test
|   └─ security-checklist.md      # Checklist bảo mật
|   └─ performance-checklist.md  # Checklist hiệu năng
|   └─ orchestration-patterns.md # Mẫu orchestration đúng/sai
└─ hooks/
|   └─ hooks.json                 # Khai báo hook được plugin dùng
|   └─ session-start.sh           # Inject meta-skill vào session mới
|   └─ sdd-cache-pre.sh           # Hook cache cho source-driven-development
|   └─ sdd-cache-post.sh          # Hook ghi/revalidate cache sau khi fetch docs
|   └─ simplify-ignore.sh         # Hook bảo vệ vùng code không được simplify
└─ docs/
    └─ getting-started.md         # Cách dùng chung cho agent bất kỳ
    └─ skill-anatomy.md           # Chuẩn cấu trúc của một SKILL.md
    └─ cursor-setup.md            # Hướng dẫn tích hợp với Cursor
    └─ gemini-cli-setup.md        # Hướng dẫn tích hợp với Gemini CLI
    └─ ...                        # Các tài liệu setup/hướng dẫn còn lại

```

(Nguồn: `README.md`, mục *Quick Start*; thư mục `references/`, `hooks/`, `docs/`)

3. Meta-Skill: `using-agent-skills` — Bộ não Điều phối

3.1. Skill Discovery Flowchart

Khi nhận task, AI phải dùng cây quyết định sau để chọn skill:

Task arrives

```

|
└─ Ý tưởng mơ hồ?           → idea-refine
└─ Dự án/tính năng mới?     → spec-driven-development
└─ Có spec, cần chia task?  → planning-and-task-breakdown
└─ Đang code?               → incremental-implementation

```

		UI?	→	frontend-ui-engineering
		API?	→	api-and-interface-design
		Cần context tốt hơn?	→	context-engineering
		Cần docs chính thức?	→	source-driven-development
		Viết/chạy test?	→	test-driven-development
		Browser-based?	→	browser-testing-with-devtools
		Có lỗi?	→	debugging-and-error-recovery
		Review code?	→	code-review-and-quality
		Bảo mật?	→	security-and-hardening
		Hiệu năng?	→	performance-optimization
		Commit/branch?	→	git-workflow-and-versioning
		CI/CD?	→	ci-cd-and-automation
		Viết docs/ADR?	→	documentation-and-adrs
		Deploy/launch?	→	shipping-and-launch

(Nguồn: `skills/using-agent-skills/SKILL.md`, dòng 16-37)

3.2. Lifecycle Sequence hoàn chỉnh

- | | |
|--------------------------------|-----------------------------------|
| 1. idea-refine | → Refine ý tưởng |
| 2. spec-driven-development | → Định nghĩa xây gì |
| 3. planning-and-task-breakdown | → Chia nhỏ thành chunks |
| 4. context-engineering | → Load context đúng lúc |
| 5. source-driven-development | → Đối chiếu docs chính thức |
| 6. incremental-implementation | → Build từng slice |
| 7. test-driven-development | → Chứng minh từng slice hoạt động |
| 8. code-review-and-quality | → Review trước merge |
| 9. git-workflow-and-versioning | → Commit history sạch |
| 10. documentation-and-adrs | → Ghi lại quyết định |
| 11. shipping-and-launch | → Deploy an toàn |

(Nguồn: `skills/using-agent-skills/SKILL.md`, dòng 133-149)

4. Hệ thống 21 Kỹ năng (The 20+1 Skills)

4.1. Giai đoạn Define (Xác định)

- `idea-refine` : Phân tích hội tụ/phân kỳ để biến ý tưởng mơ hồ thành đề xuất cụ thể.

- `spec-driven-development` : Viết PRD gồm mục tiêu, commands, cấu trúc, code style, testing strategy, boundaries. **Trước khi** viết code. (Nguồn: `README.md`, dòng 134-137)

4.2. Giai đoạn Plan (Kế hoạch)

- `planning-and-task-breakdown` : Chẻ nhỏ Spec thành task có acceptance criteria và dependency ordering. (Nguồn: `README.md`, dòng 141-143)

4.3. Giai đoạn Build (Xây dựng)

- `incremental-implementation` : Cắt lát dọc (Thin vertical slices) — code, test, verify, commit từng phần. Feature flags, safe defaults.
- `context-engineering` : Đóng gói thông tin đúng lúc cho AI — rules files, context packing, MCP integrations.
- `source-driven-development` : Ép mọi quyết định framework phải dựa trên docs chính thức (verified citations).
- `frontend-ui-engineering` : Kiến trúc Component, design systems, state management, WCAG 2.1 AA accessibility.
- `api-and-interface-design` : Contract-first design, Hyrum's Law, One-Version Rule, error semantics. (Nguồn: `README.md`, dòng 147-154)

4.4. Giai đoạn Verify (Kiểm chứng)

- `test-driven-development` : Red-Green-Refactor, kim tự tháp Test (80% unit / 15% integration / 5% E2E), quy tắc Beyonce, DAMP over DRY.
- `browser-testing-with-devtools` : Chrome DevTools MCP — DOM, console logs, network traces, performance profiling.
- `debugging-and-error-recovery` : Triage 5 bước: Reproduce → Localize → Reduce → Fix → Guard. (Nguồn: `README.md`, dòng 147-161; `skills/using-agent-skills/SKILL.md`, dòng 165-167)

4.5. Giai đoạn Review (Đánh giá)

- `code-review-and-quality` : Review 5 trục, severity labels (Nit/Optional/FYI), kích thước lý tưởng (~100 dòng).
- `code-simplification` : Nguyên tắc Chesterton's Fence, Rule of 500, rút gọn không đổi hành vi.
- `security-and-hardening` : OWASP Top 10, auth patterns, secrets management, three-tier boundary.
- `performance-optimization` : Measure-first, Core Web Vitals targets, bundle analysis. (Nguồn: `README.md`, dòng 163-170)

4.6. Giai đoạn Ship (Triển khai)

- `git-workflow-and-versioning` : Trunk-based dev, atomic commits, commit-as-save-point pattern.
- `ci-cd-and-automation` : Shift Left, Faster is Safer, feature flags, quality gate pipelines.
- `deprecation-and-migration` : Code-as-liability, compulsory vs advisory deprecation, zombie code removal.
- `documentation-and-adrs` : Architecture Decision Records, inline docs — document the *why*.
- `shipping-and-launch` : Pre-launch checklists, staged rollouts, rollback procedures, monitoring.
(Nguồn: `README.md`, dòng 174-180)

4.7. Meta-Skill

- `using-agent-skills` : Kỹ năng điều phối — chứa Skill Discovery Flowchart, 6 Core Behaviors, 10 Failure Modes. Được inject tự động vào mọi session qua hook `session-start.sh`. (Nguồn: `skills/using-agent-skills/SKILL.md`, dòng 1-175; `hooks/session-start.sh`)

5. Giải phẫu một Kỹ năng (Skill Anatomy)

Mọi `SKILL.md` tuân thủ chuẩn thống nhất:

1. **Frontmatter (YAML):** `name` (kebab-case, khớp tên thư mục) + `description` (tối đa 1024 ký tự, bao gồm "Use when..." trigger).
2. **Overview:** Elevator pitch — skill này làm gì, tại sao cần theo.
3. **When to Use:** Trigger conditions + exclusions (khi NÀO không dùng).
4. **Core Process:** Workflow bước-bước. **Specific over general** — "Run `npm test`" thay vì "make sure tests work".
5. **Common Rationalizations:** Bảng nguy biện AI + cách phản bác.
6. **Red Flags:** Dấu hiệu tiến trình đang sai hướng.
7. **Verification:** Checklist exit criteria — mỗi checkbox phải có bằng chứng.

6 Writing Principles:

- Process over knowledge | Specific over general | Evidence over assumption
- Anti-rationalization | Progressive disclosure | **Token-conscious** (section nào không ảnh hưởng hành vi AI → xóa)

Quy tắc file hỗ trợ: Chỉ tạo khi vượt 100 dòng. References đặt ở `references/`, KHÔNG trong thư mục skill. (Nguồn: `docs/skill-anatomy.md`, dòng 1-129; `CONTRIBUTING.md`, dòng 12-43)

6. Phân tích 7 Lệnh (Commands Deep Dive)

6.1. `/spec` — Xác định Yêu cầu

1. AI hỏi user về 4 khía cạnh: Objective, Core features, Tech stack, Boundaries.
2. Tạo `SPEC.md` gồm 6 phần cấu trúc.
3. **Dừng chờ User phê duyệt** mới tiếp tục. (Nguồn: `.claude/commands/spec.md`, dòng 5-15)

6.2. `/plan` — Chia nhỏ Kế hoạch

1. AI ở chế độ **read-only** — không được sửa code.
2. Vẽ Dependency Graph giữa components.
3. Cắt theo **Vertical Slices** (một tính năng hoàn chỉnh end-to-end).
4. Xuất `tasks/plan.md` và `tasks/todo.md`. (Nguồn: `.claude/commands/plan.md`, dòng 5-16)

6.3. `/build` — Xây dựng Tĩnh tiến

1. Nhật task từ todo. Đọc acceptance criteria.
2. **RED**: Viết test thất bại. **GREEN**: Code tối thiểu cho pass.
3. Chạy regression test + build.
4. Commit, đánh dấu hoàn thành, chuyển task tiếp.
5. Nếu fail → tự động gọi `debugging-and-error-recovery`. (Nguồn: `.claude/commands/build.md`, dòng 5-18)

6.4. `/test` — Kiểm chứng

- **Tính năng mới**: RED → GREEN → Refactor.
- **Bug fix (Prove-It pattern)**: (1) Viết test tái hiện bug (FAIL) → (2) Xác nhận fail → (3) Fix → (4) Xác nhận PASS → (5) Chạy full suite.
- Browser: tích hợp thêm Chrome DevTools MCP. (Nguồn: `.claude/commands/test.md`, dòng 5-19)

6.5. `/review` — Đánh giá 5 Trục

1. **Correctness** — Đúng spec? Edge cases? Tests đủ?
2. **Readability** — Tên biến chuẩn? Logic rõ ràng?
3. **Architecture** — Đúng pattern? Ranh giới sạch?
4. **Security** — Input validated? Secrets safe?
5. **Performance** — N+1 queries? Unbounded ops?

Phân loại: **Critical** (phải sửa) | **Important** (nên sửa) | **Suggestion** (cân nhắc). Kèm `file:line` cụ thể.
(Nguồn: `.claude/commands/review.md`, dòng 5-16)

6.6. /code-simplify — Đơn giản hóa

- 1. Đọc project conventions, hiểu mục đích code, callers, edge cases.
- 2. Nhận diện: Deep nesting → Guard clauses; Hàm dài → Split; Duplicate → Shared; Dead code → Remove.
- 3. Áp dụng từng bước một — chạy test sau mỗi thay đổi.
- 4. Test FAIL → Revert ngay lập tức. (Nguồn: .claude/commands/code-simplify.md, dòng 5-22)

6.7. /ship — Fan-out Orchestrator

- 1. Phase A (Parallel fan-out): Mở song song 3 sub-agents: code-reviewer, security-auditor, test-engineer. Chúng đánh giá độc lập, không giao tiếp with nhau.
- 2. Phase B (Merge): AI chính tổng hợp 3 báo cáo + kiểm tra thêm Accessibility, Infrastructure, Documentation.
- 3. Phase C (Decision): Quyết định GO hoặc NO-GO. Nếu GO, bắt buộc kèm Rollback Plan (trigger conditions, procedure, recovery time objective).

Ngoại lệ: Bỏ qua fan-out nếu tất cả điều kiện sau đúng: ≤2 files, <50 dòng diff, không chạm auth/payments/data/config. (Nguồn: .claude/commands/ship.md, dòng 5-73)

7. Hệ thống Personas (Specialist Agents)

7.1. Ba Persona Chuyên gia

Persona	Vai trò	Góc nhìn
code-reviewer	Senior Staff Engineer	Review 5 trục, verdict APPROVE/REQUEST CHANGES
security-auditor	Security Engineer	OWASP, threat modeling, CVE, auth/authz
test-engineer	QA Specialist	Test strategy, coverage analysis, Prove-It pattern
(Nguồn: README.md, dòng 188-192; agents/code-reviewer.md)		

7.2. Composition Rules (Quy tắc Phối hợp)

Mỗi persona kết thúc bằng block ## Composition quy định rõ:

- **Gọi trực tiếp khi:** User yêu cầu review/audit/test cụ thể.

- **Gọi qua command:** `/review`, `/ship`.
- **KHÔNG BAO GIỜ gọi từ persona khác.** Nếu `code-reviewer` thấy cần security audit → chỉ **recommend** trong báo cáo, không tự gọi `security-auditor`.

Ràng buộc này được Claude Code **cưỡng chế ở cấp nền tảng**: subagents cannot spawn subagents.
(Nguồn: `agents/code-reviewer.md`, dòng 93-97; `agents/README.md`, dòng 96-101)

8. Orchestration Patterns — Mẫu Điều phối

8.1. Năm Mẫu Được Chấp nhận

#	Pattern	Cost	Ví dụ
1	Direct invocation — 1 persona, 1 artifact	1 round trip	"Review PR này" → <code>code-reviewer</code>
2	Single-persona command — Wrap 1 persona + skill	= Direct	<code>/review</code> , <code>/test</code>
3	Parallel fan-out + merge — N personas song song	N contexts + 1 merge	<code>/ship</code>
4	Sequential pipeline — User là orchestrator	1 context/step	<code>/spec</code> → <code>/plan</code> → <code>/build</code> → <code>/test</code> → <code>/review</code> → <code>/ship</code>
5	Research isolation — Sub-agent đọc nhiều, trả digest nhỏ	1 isolated context	"Tìm mọi call site của deprecated API"
(Nguồn: <code>references/orchestration-patterns.md</code> , dòng 9-113)			

8.2. Bốn Anti-patterns Bị Cấm

- **A. Router persona** — Persona "meta-orchestrator" chỉ routing, không có domain value. Tốn 2× token.
- **B. Persona gọi persona** — Mất context, nhân failure modes, ẩn cost.
- **C. Sequential orchestrator tự paraphrase** — Mất human checkpoints, drift tích lũy.
- **D. Deep persona trees** — Nhiều tầng = latency + token vô ích. (Nguồn: `references/orchestration-patterns.md`, dòng 282-339)

8.3. Sub-agents vs Agent Teams

	Sub-agents (/ship)	Agent Teams (thử nghiệm)
Giao tiếp	Chỉ báo cáo ngược cho Agent chính	Teammates nhắn tin trực tiếp với nhau
Mục đích	Đưa phán quyết (verdict) trên artifact đã biết	Điều tra qua tranh luận đối kháng (competing hypotheses)
Khi dùng	PR review, pre-deploy check	Debug lỗi Production phức tạp
Trạng thái	Stable	Experimental (CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1)
(Nguồn: <code>references/orchestration-patterns.md</code> , dòng 125-278)		

9. Hooks System — Vòng đời Session

9.1. Session Start Hook

Mỗi khi Agent bắt đầu session mới, `hooks/session-start.sh` tự động inject nội dung meta-skill `using-agent-skills` vào context. Đảm bảo Agent luôn biết cách chọn đúng skill ngay từ đầu. (Nguồn: `hooks/hooks.json`; `hooks/session-start.sh`, dòng 1-25)

9.2. SDD-Cache (Source-Driven Development Cache)

Cache HTTP thông minh cho `source-driven-development`:

- Cache nội dung docs đã fetch lên đĩa (`.claude/sdd-cache/<sha>.json`).
- Luôn **revalidate** với server gốc qua `ETag / If-Modified-Since` .
- Chỉ serve cache khi server trả `304 Not Modified` — **không bao giờ trả docs cũ**.
- Server không có ETag/Last-Modified → **không cache**.

Yêu cầu: `jq`, `curl`, `shasum`, Bash 3.2+. (Nguồn: `hooks/SDD-CACHE.md`, dòng 1-168)

9.3. Simplify-Ignore Hook

Cho phép đánh dấu file/đoạn code không được phép simplify, bảo vệ code có chủ đích phức tạp. (Nguồn: `hooks/SIMPLIFY-IGNORE.md`)

10. Tương thích Đa nền tảng (Multi-platform)

10.1. Cài đặt với Cursor

`README.md` chỉ nêu ngắn gọn rằng có thể copy `SKILL.md` vào `.cursor/rules/` hoặc trỏ tới toàn bộ thư mục `skills/` (`README.md`, dòng 63-68). Tài liệu chi tiết hơn nằm ở `docs/cursor-setup.md`.

Cách cài khuyến nghị:

1. Tạo thư mục `.cursor/rules/` trong project.
2. Copy các skill quan trọng vào đây, ví dụ:
 - `test-driven-development`
 - `code-review-and-quality`
 - `incremental-implementation`
3. Để các skill dùng theo ngữ cảnh ở dạng Notepad thay vì load tất cả cùng lúc.

Ví dụ lệnh:

```
mkdir -p .cursor/rules
cp /path/to/agent-skills/skills/test-driven-development/SKILL.md .cursor/rules/test-dr
cp /path/to/agent-skills/skills/code-review-and-quality/SKILL.md .cursor/rules/code-re
cp /path/to/agent-skills/skills/incremental-implementation/SKILL.md .cursor/rules/incr
```

Lựa chọn khác:

- Dùng file `.cursorrules` để inline vài skill thiết yếu.
- Dùng Notepads của Cursor để lưu các skill theo từng phase và gọi lại bằng `@notepad`.

Lưu ý thực tế: không nên load toàn bộ skill cùng lúc vì Cursor có giới hạn context; cách hợp lý là giữ 2-3 skill lõi trong rules, còn lại chỉ nạp khi cần. (Nguồn: `README.md`, dòng 63-68; `docs/cursor-setup.md`)

10.2. Cài đặt với Gemini CLI

`README.md` cung cấp hướng dẫn cài khá rõ cho Gemini CLI (`README.md`, dòng 70-87). Ý chính là cài `agent-skills` như **native skills** để Gemini tự discover, hoặc thêm vào `GEMINI.md` nếu muốn giữ persistent context.

Cài trực tiếp từ repo GitHub:

```
gemini skills install https://github.com/addyosmani/agent-skills.git --path skills
```

Cài từ bản clone local:

```
gemini skills install ./agent-skills/skills/
```

Sau khi cài, Gemini CLI có thể tự nhận diện bộ skills và ánh xạ task vào skill phù hợp. Repo cũng có thư mục `.gemini/commands/` để cung cấp bộ command tương ứng cho môi trường Gemini. (Nguồn: `README.md`, dòng 70-87; thư mục `.gemini/commands/`)

10.3. Các nền tảng khác

Với các nền tảng còn lại, báo cáo này chỉ ghi đường dẫn tra cứu nhanh trong `README.md`:

- **Claude Code:** xem `README.md`, dòng 38-61.
- **Windsurf:** xem `README.md`, dòng 89-94.
- **OpenCode:** xem `README.md`, dòng 96-103.
- **GitHub Copilot:** xem `README.md`, dòng 105-110.
- **Kiro IDE & CLI:** xem `README.md`, dòng 112-115.
- **Codex / Other Agents:** xem `README.md`, dòng 117-122.

Bản chất chung: skills là các file Markdown, nên ngoài các tích hợp chính thức ở trên, bộ KIT vẫn dùng được với bất kỳ agent nào hỗ trợ system prompts hoặc rules files. (Nguồn: `README.md`, mục *Quick Start*; `docs/getting-started.md`)

11. Tổng kết Đánh giá

11.1. Ưu điểm (Pros)

- **Chất lượng Senior Engineer:** Code được bảo hành qua TDD, OWASP review, 5-axis code review.
- **Tính nhất quán:** AI không "ảo giác" nhờ workflow bắt buộc + anti-rationalization tables.
- **An toàn khi Deploy:** `/ship` fan-out 3 chuyên gia + bắt buộc Rollback Plan.
- **Tiết kiệm Token:** Progressive disclosure — skill chỉ load khi cần, `SKILL.md` dưới 500 dòng.
- **Đa nền tảng thực dụng:** Claude Code, Gemini CLI, Cursor, OpenCode, Windsurf, Copilot, Kiro đều được `README` hoặc docs nhắc cách dùng ở các mức độ khác nhau. (Nguồn: `README.md`; `docs/`)

11.2. Nhược điểm (Cons)

- **Chi phí context/token cao hơn cho task lớn:** Đây là nhận định phân tích. Chu trình `/spec` → `/plan` → `/build` và fan-out `/ship` tạo thêm artifact, thêm lượt suy luận, thêm context window so với việc prompt trực tiếp.
- **Có độ cứng quy trình với task nhỏ:** Đây cũng là nhận định phân tích. Với các thay đổi rất nhỏ, việc luôn nghĩ theo spec, task breakdown, verification và review có thể bị cảm nhận là nặng tay, dù `/ship` cho phép bỏ qua fan-out với diff nhỏ.
- **Hooks cần Bash + jq:** SDD-Cache và session-start yêu cầu Bash 3.2+, `jq`, `curl` — không thân thiện trên Windows thuần. (Nguồn: `.claude/commands/ship.md`, phần `rules`; `hooks/SDD-CACHE.md`, phần `requirements`; hai ý đầu là phân tích tổng hợp của người viết)

11.3. Kịch bản Áp dụng Tối ưu (Use Cases)

1. **Dự án Mới / Module phức tạp:** `/spec` → `/plan` → `/build` để AI hiểu toàn cảnh trước khi code.
2. **Legacy Code Refactoring:** `/code-simplify` với Chesterton's Fence + test liên tục sau mỗi thay đổi.
3. **Debug lỗi Production:** `/test` với Prove-It pattern. Bug cần competing hypotheses → Agent Teams.
4. **Pre-deploy tính năng nhạy cảm:** `/ship` để 3 sub-agents quét OWASP, coverage gaps, architectural issues.
5. **Migration hệ thống cũ:** Skill `deprecation-and-migration` + `/code-simplify` để dọn zombie code. (Nguồn: Phân tích tổng hợp từ toàn bộ KIT)